

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0501
NAME: M.NITHYASHREENARAYANI
REGNO: 192521416

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 30%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.	BL3 – Apply	5
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.	BL3 – Apply	5
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.	BL3 – Apply	5
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation.	BL3 – Apply	5
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.	BL3 – Apply	5
6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.	BL6 – Create	5
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.	BL3 – Apply	5
8	Identify and fix the logical error in a given SQL UPDATE statement	BL3 – Apply	5

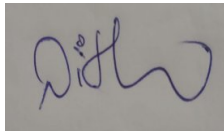
	that unintentionally modifies all rows in the table.		
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.	BL6 – Create	5
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.	BL3 – Apply	5

Code of Conduct:

I M.NITHYASHREENARAYANI / 192521416 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained

		algorithms			
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

DATABASE MANAGEMENT SYSTEMS

SQL Debugging, Optimization & Relational Model — Assessment Answer Key

Total Marks: 50 | 10 Questions × 5 Marks

Reference Schema Used in Solutions

All queries below are illustrated on the following common schema so that each answer is self-contained and testable:

```
Departments (DeptID PK, DeptName, Location)
Employees (EmpID PK, EmpName, DeptID FK->Departments, Salary, ManagerID)
Projects (ProjID PK, ProjName, DeptID FK->Departments, Budget)
EmpProject (EmpID FK->Employees, ProjID FK->Projects, HoursWorked)
```

Each answer follows the same structure: the faulty statement, identification of the error, the corrected/optimized statement, and a short explanation — matching the Apply/Create Bloom's levels indicated.

Q1. Incorrect JOIN Syntax Causing Duplicate Results

Bloom's Level: BL3 – Apply Marks: 5

Faulty Query

```
SELECT E.EmpName, D.DeptName
FROM Employees E, Departments D;
```

Identification of Error

This uses the old-style comma (implicit) join without any WHERE/ON condition linking the two tables. SQL therefore computes a Cartesian product — every employee row is paired with every department row. If there are 50 employees and 5 departments, the result set has 250 rows instead of 50, most of them meaningless duplicates/mismatches.

Corrected Query

```
SELECT E.EmpName, D.DeptName
FROM Employees E
INNER JOIN Departments D
ON E.DeptID = D.DeptID;
```

Explanation

Using an explicit INNER JOIN with an ON clause forces the engine to match rows only where E.DeptID = D.DeptID, eliminating the Cartesian product. Each employee now appears exactly once, joined to their correct department — no duplicates, correct row count.

Q2. Sub-query With Missing Correlation

Bloom's Level: BL3 – Apply *Marks: 5*

Requirement

List employees who earn more than the average salary of their own department.

Faulty Query

```
SELECT EmpName, Salary, DeptID
FROM Employees E
WHERE Salary > (SELECT AVG(Salary) FROM Employees);
```

Identification of Error

The inner sub-query `SELECT AVG(Salary) FROM Employees` computes ONE single average across the entire company, because it never references the outer query's `DeptID`. It is an uncorrelated sub-query being used where a correlated one is required, so every employee is compared to the company-wide average instead of their own department's average — producing an incorrect employee list.

Corrected Query

```
SELECT EmpName, Salary, DeptID
FROM Employees E
WHERE Salary > (
    SELECT AVG(E2.Salary)
    FROM Employees E2
    WHERE E2.DeptID = E.DeptID
);
```

Explanation

Adding the condition `E2.DeptID = E.DeptID` correlates the inner query with the outer row, so the sub-query re-evaluates the average separately for each employee's own department. This produces the intended per-department comparison.

Q3. Faulty View Definition

Bloom's Level: BL3 – Apply *Marks: 5*

Faulty View

```
CREATE VIEW DeptSalarySummary AS
SELECT DeptID, EmpName, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY DeptID;
```

Identification of Error

`EmpName` is selected but is neither part of the `GROUP BY` clause nor wrapped in an aggregate function. In standard SQL (and MySQL's `ONLY_FULL_GROUP_BY` mode) this raises an error; in lenient modes it silently returns an arbitrary `EmpName` per group, making the view's output non-deterministic and unreliable for reporting.

Corrected & Optimized View

```
CREATE VIEW DeptSalarySummary AS
SELECT D.DeptID,
       D.DeptName,
       COUNT(E.EmpID) AS EmployeeCount,
       AVG(E.Salary) AS AvgSalary,
       MAX(E.Salary) AS MaxSalary
FROM Departments D
LEFT JOIN Employees E ON D.DeptID = E.DeptID
GROUP BY D.DeptID, D.DeptName;
```

Explanation

Every non-aggregated column (DeptID, DeptName) is now part of GROUP BY, and EmpName has been replaced with meaningful aggregates (COUNT, AVG, MAX). The LEFT JOIN from Departments also ensures departments with zero employees still appear in the summary (EmployeeCount = 0) instead of being silently dropped, which is both correct and more useful for reporting.

Q4. Faulty Relational Algebra Expression for Natural Join

Bloom's Level: BL3 – Apply *Marks: 5*

Schema Attributes

Employees(EmpID, EmpName, Dept_ID, Salary) Departments(DeptID, DeptName)

Faulty Expression

```
Employees ⋈ Departments
```

Identification of Error

Natural join ($\bowtie$) automatically equates attributes that share the SAME NAME across both relations. Here the foreign key is named Dept_ID in Employees but DeptID in Departments — the names don't match. Since no common attribute name exists, the natural join degenerates into a plain Cartesian product (σ is never applied), producing incorrect, bloated output instead of the intended department-wise join.

Corrected Expression

Option 1 — rename the attribute so natural join can apply correctly:

```
 $\rho$ (Employees(EmpID, EmpName, DeptID, Salary), Employees) ⋈ Departments
```

Option 2 — use an explicit theta join instead of relying on name matching:

```
Employees ⋈ (Employees.Dept_ID = Departments.DeptID) Departments
```

Explanation

Rename (ρ) makes the join attribute names identical so the natural join correctly matches tuples on department. Alternatively, a theta join ($\bowtie\theta$) explicitly states the join predicate, bypassing the name-matching rule entirely. Either fix removes the accidental Cartesian product.

Q5. Referential Integrity Violation

Bloom's Level: BL3 – Apply **Marks:** 5

Faulty Table Definitions & Statement

```
CREATE TABLE Employees (
  EmpID INT PRIMARY KEY,
  EmpName VARCHAR(50),
  DeptID INT
);

INSERT INTO Employees (EmpID, EmpName, DeptID)
VALUES (101, 'Ravi', 50); -- DeptID 50 does not exist in Departments
```

Identification of Violation

DeptID in Employees is a logical foreign key to Departments.DeptID, but no FOREIGN KEY constraint was ever declared. Consequently the database happily inserts EmpID 101 with DeptID = 50 even though no such department exists, creating an orphan record. This breaks referential integrity — every child value must correspond to an existing parent value.

Corrected Statements

```
CREATE TABLE Employees (
  EmpID INT PRIMARY KEY,
  EmpName VARCHAR(50),
  DeptID INT,
  CONSTRAINT FK_Emp_Dept FOREIGN KEY (DeptID)
    REFERENCES Departments(DeptID)
    ON UPDATE CASCADE
    ON DELETE SET NULL
);

-- This insert now fails and is rejected by the engine:
INSERT INTO Employees (EmpID, EmpName, DeptID)
VALUES (101, 'Ravi', 50); -- ERROR: FK constraint violation

-- Valid insert, since DeptID 10 exists in Departments:
INSERT INTO Employees (EmpID, EmpName, DeptID)
VALUES (101, 'Ravi', 10);
```

Explanation

Declaring the FOREIGN KEY constraint forces DeptID to reference an existing Departments.DeptID (or be NULL). ON DELETE SET NULL / ON UPDATE CASCADE define safe, controlled behaviour when a parent row changes, instead of silently allowing dangling references.

Q6. Rewriting a Nested-Loop Query Using JOIN (Optimization)

Bloom's Level: BL6 – Create **Marks:** 5

Requirement

List the names of employees who belong to a department located in 'Chennai'.

Inefficient Query (row-by-row / nested sub-query style)

```
SELECT EmpName
FROM Employees
WHERE DeptID IN (
    SELECT DeptID FROM Departments WHERE Location = 'Chennai'
);
```

Why It Is Inefficient

Written this way, many engines evaluate the sub-query once per outer row (a nested-loop pattern) rather than performing a single set-based join, which does not scale well as both tables grow and cannot benefit from a hash/merge join plan.

Optimized JOIN-Based Query

```
SELECT DISTINCT E.EmpName
FROM Employees E
INNER JOIN Departments D
    ON E.DeptID = D.DeptID
WHERE D.Location = 'Chennai';
```

Explanation

Rewriting the IN sub-query as an INNER JOIN lets the query optimizer choose an efficient set-based strategy (hash join or merge join) instead of a per-row lookup. Combined with an index on Departments(Location) and Employees(DeptID), this scales far better on large tables.

Q7. GROUP BY Query Producing Wrong Aggregation

Bloom's Level: BL3 – Apply *Marks: 5*

Faulty Query

```
SELECT DeptID, EmpName, SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY DeptID;
```

Identification of Error

EmpName is a non-aggregated, non-grouped column. GROUP BY DeptID collapses all rows of a department into ONE group, but SQL has no way to pick a single, meaningful EmpName to represent that whole group. In strict-mode engines this is a syntax/semantic error; in lenient engines it silently returns an arbitrary employee's name next to the correctly summed salary, giving a misleading result that looks valid but is not.

Corrected Query

If only the department-level total is needed:

```
SELECT DeptID, SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY DeptID;
```

If per-employee detail must be retained alongside the departmental total, use a window function instead of GROUP BY:


```
SELECT DeptID, EmpName, Salary,
       SUM(Salary) OVER (PARTITION BY DeptID) AS DeptTotalSalary
FROM Employees;
```

Explanation

Removing EmpName fixes the aggregation error directly. The window-function alternative shows how to keep row-level detail while still exposing a group aggregate, which GROUP BY alone cannot do.

Q8. Logical Error in UPDATE Statement Modifying All Rows

Bloom's Level: BL3 – Apply Marks: 5

Faulty Statement

```
-- Intent: give a 10% raise to employees in Department 10 only
UPDATE Employees
SET Salary = Salary * 1.10;
```

Identification of Error

The statement has no WHERE clause. An UPDATE (like a DELETE) with no WHERE clause is applied to every row in the table, so every employee across every department receives the raise, not just Department 10 as intended. This is one of the most common and costly SQL mistakes.

Corrected Statement

```
UPDATE Employees
SET Salary = Salary * 1.10
WHERE DeptID = 10;
```

Explanation

Adding WHERE DeptID = 10 restricts the update to only the intended rows. As good practice, such updates should first be verified with a SELECT using the identical WHERE clause (and ideally run inside a transaction with COMMIT after checking ROW_COUNT) before executing the UPDATE, to avoid unintended mass modification.

Q9. Optimizing a Multi-Table Query With Nested Sub-queries

Bloom's Level: BL6 – Create Marks: 5

Requirement

List distinct employee names who work in a department located in 'Chennai' AND are assigned to a project with Budget > 100000.

Unoptimized Query (nested sub-queries)

```
SELECT EmpName
FROM Employees
WHERE DeptID IN (
    SELECT DeptID FROM Departments WHERE Location = 'Chennai'
)
```

```

AND EmpID IN (
    SELECT EmpID FROM EmpProject
    WHERE ProjID IN (
        SELECT ProjID FROM Projects WHERE Budget > 100000
    )
);

```

Restructured, Optimized Query

```

SELECT DISTINCT E.EmpName
FROM Employees    E
JOIN Departments  D ON E.DeptID = D.DeptID
JOIN EmpProject   EP ON E.EmpID = EP.EmpID
JOIN Projects     P ON EP.ProjID = P.ProjID
WHERE D.Location = 'Chennai'
AND P.Budget > 100000;

```

Supporting Indexes

```

CREATE INDEX idx_dept_location ON Departments(Location);
CREATE INDEX idx_emp_deptid    ON Employees(DeptID);
CREATE INDEX idx_empproj_empid ON EmpProject(EmpID);
CREATE INDEX idx_empproj_projid ON EmpProject(ProjID);
CREATE INDEX idx_proj_budget   ON Projects(Budget);

```

Explanation

Three nested IN sub-queries force the optimizer into repeated, layered row-by-row filtering. Converting them into a single chain of INNER JOINs lets the optimizer build one efficient execution plan (e.g., hash joins) across all four tables in a single pass. The added indexes on the join columns (DeptID, EmpID, ProjID) and the filter columns (Location, Budget) allow the engine to use index seeks/lookups instead of full table scans, which is critical for performance as table sizes grow. DISTINCT is retained because the join can legitimately produce repeated EmpNames when an employee has multiple qualifying projects.

Q10. Relational Calculus Expression Returning an Empty Result (Incorrect Quantifier)

Bloom's Level: BL3 – Apply *Marks: 5*

Requirement

Find employees who are assigned to at least one project.

Faulty Tuple Relational Calculus Expression

```

{ e | e ∈ Employees ∧
    ∀ p ( p ∈ Projects ⇒
        ∃ ep ( ep ∈ EmpProject ∧ ep.EmpID = e.EmpID
              ∧ ep.ProjID = p.ProjID ) ) }

```

Identification of Error

The universal quantifier $\forall p$ is used over the entire Projects relation. As written, this asks for employees who are assigned to EVERY project in the company (a division/'for-all' query), not employees assigned to at least

one project. Since it is extremely unlikely that any single employee works on literally all projects, this expression returns an empty result set — the quantifier used does not match the intended semantics.

Corrected Expression

```
{ e | e ∈ Employees ∧  
  ∃ ep ( ep ∈ EmpProject ∧ ep.EmpID = e.EmpID ) }
```

Explanation

Replacing the universal quantifier ($\forall$) with an existential quantifier ($\exists$) correctly expresses "there exists at least one EmpProject tuple for this employee." This matches the intended requirement and returns the non-empty, correct set of employees who work on one or more projects. As a general rule: use $\exists$ for 'at least one' requirements and reserve $\forall$ genuinely for 'for all / every' requirements, since swapping the two is the most common cause of unexpectedly empty relational-calculus results.